

DOCUMENT 12

Learning System Guide

Failure detection, candidate generation, approval workflow, and safety gates

Overview

The NGW learning system monitors live user outcomes, detects patterns of failure, and automatically generates typed improvement proposals called Candidates. Every proposal begins in "proposed" status. Nothing is auto-implemented — all changes require explicit curator approval. Seeded and internal signals are excluded.

Live only

Signal source for learning

5 types

Failure cluster categories

Human req.

All changes need approval

0

Auto-implemented changes

Data Requirements

Parameter	Value	Notes
Minimum signal count	≥ 30	Per pattern per time window
Time window	30 days	Rolling; configurable via env
Signal source filter	live only	include_in_learning = TRUE AND source = live
Minimum cluster size	≥ 10	Failures in same category to form a cluster
Severity threshold	≥ 0.25	Normalized score; below this = not surfaced

Failure Detection Algorithm

engine/learning/auto_candidate.py — detection loop

```
# Runs on schedule (weekly) or triggered manually.
def detect_failures(db, window_days=30, min_n=30):
    signals = db.query(
        "SELECT pattern, outcome, reliability_score, gear_tier"
        " FROM session_signals"
        " WHERE include_in_learning = TRUE"
        "   AND signal_source = 'live'"
        "   AND created_at >= DATE('now', '-%d days')" % window_days
    )
    by_pattern = group_by(signals, "pattern")
    clusters = []
    for pattern, rows in by_pattern.items():
        if len(rows) < min_n:
            continue
        failure_rate = count(rows, outcome='failed') / len(rows)
        cvr = count(rows, outcome__in=['nailed_it', 'close']) / len(rows)
        cluster = FailureCluster(
            pattern=pattern, n=len(rows),
            failure_rate=failure_rate, cvr=cvr
        )
        clusters.append(cluster)
    return clusters
```

Severity Scoring

Each failure cluster is scored on a 0–1 scale. Clusters above the threshold (0.25) are promoted to Candidates automatically. The formula rewards large clusters and penalizes high-volume patterns with low failure rates.

engine/learning/auto_candidate.py — severity formula

```
def severity(cluster: FailureCluster) -> float:
    # Volume component: log-scaled, capped at 500 signals
    volume = min(cluster.n, 500) / 500
    # Failure rate component: straight failure rate
    fail_r = cluster.failure_rate
    # CVR gap: how far below 0.60 baseline CVR
    cvr_gap = max(0, 0.60 - cluster.cvr) / 0.60
    # Weighted sum
    return 0.35 * volume + 0.35 * fail_r + 0.30 * cvr_gap
```

Failure-to-Candidate Type Mapping

Failure Pattern	Candidate Type	Trigger Condition
conversion_gap	blueprint_correction	CVR < 25% with n ≥ 50
confidence_mismatch	confidence_recalibration	High confidence but failure_rate > 40%
step_deviation	shoot_mode_step_fix	Users consistently skip or fail one step
pattern_drift	dataset_promotion	Pattern accuracy declining over 4 weeks
trust_gap	trust_safety	High-confidence wrong results ≥ 3 weeks

Auto-Generated Candidate Example

Example: auto-generated blueprint_correction candidate (JSON)

```
{
  "id": "cand_uuid_here",
  "title": "[Auto] Conversion gap – 'loop' pattern (n=150, CVR=18%)",
  "description": "Loop pattern detected in 150 analyses but yielded
    only 27 upgrades (18% CVR). Engine may be over-diagnosing this
    pattern in low-ceiling environments.",
  "candidate_type": "blueprint_correction",
  "status": "proposed",
  "proposed_change": {
    "type": "blueprint_correction",
    "action": "review_detection_threshold",
    "review_areas": ["threshold", "blueprint_copy", "confidence_gate"],
    "affected_pattern": "loop"
  },
  "estimated_success_lift": 0.22,
  "estimated_regression_risk": 0.08,
  "source": "auto"
}
```

Approval Workflow

Auto-Generate

1

Learning sweep detects cluster creates candidate with status="proposed".

Curator Review

2

Lab curator reads description, proposed_change, lift, and regression risk.

Risk Assessment

3

High risk (regression_risk > 0.20) requires a second curator approval.

4 Approve/Reject

Curator sets status="approved" or status="rejected" with notes.

5 Engine Update

Approved candidate is applied to blueprint or detection config.

6 Validation

Gold set evaluation re-runs automatically. Any regression reverts the change.

Engine Update & Rollback

bash — apply and rollback a candidate

```
# Apply an approved candidate
$ python3 scripts/apply_candidate.py --id cand_uuid_here

# Re-run gold set evaluation to confirm no regression
$ python3 scripts/run_benchmarks.py --gold-sets-only

# Rollback: revert candidate to proposed status
$ sqlite3 data/ngw_users.db \
  "UPDATE candidates SET status='proposed' WHERE id='cand_uuid_here';"

# Then manually revert the blueprint change and re-run gold sets
```

REGRESSION SAFETY GATE

- Gold set evaluation is mandatory after every approved candidate. If any gold set score drops below 75%, the deployment pipeline fails and the candidate is automatically reverted to "proposed" status.
- There is no way to bypass this gate in production. Local dev only.

Learning Performance Benchmarks

Pattern	Baseline CVR	Target Post-Update	Min Signals Needed	Notes
Clamshell	85–92%	90–95%	30	Incremental gains only
Loop	60–80%	72–85%	30	Most improvement headroom
Rembrandt	40–70%	55–75%	50	Complex; high variance
Butterfly	70–82%	75–87%	30	Ceiling height key factor
Split	65–78%	68–82%	30	Gear tier dominant factor
Hard Direct	55–70%	65–78%	40	Needs modifier data